

ADC Temperature Monitor

STM32L476RG Internal Sensor, ADC1, DMA, MFS Display, and UART Product
Documentation

Project documentation staged in docs/

May 14, 2026

Contents

1	Executive Summary	3
2	Source of Truth	3
3	Repository and Build Context	3
4	Black-Box Product Contract	4
4.1	Inputs	4
4.2	Outputs	4
5	Hardware Architecture	4
5.1	Board and MCU	4
5.2	ADC Signal Path	5
5.3	Display Interface	5
5.4	UART Interface	5
6	Firmware Architecture	5
6.1	Module Responsibilities	5
6.2	Initialization Sequence	6
6.3	ADC and DMA Configuration	6
7	Measurement and Calibration Equations	6
7.1	Averaging	6
7.2	Analog Supply Estimation	7
7.3	Temperature Code Compensation	7
7.4	Two-Point Temperature Interpolation	7
8	Runtime Behavior	7
8.1	Main Loop Cadence	7
8.2	Fault Model	8
9	Connection Checklist	8
10	Official Hardware Evidence	9
10.1	NUCLEO-L476RG Arduino Connector Evidence	10
10.2	MB1136 Schematic Evidence	12
10.3	STM32L476xx Internal ADC Evidence	14

11 System Diagrams	16
11.1 Product Context	16
11.2 ADC and DMA Pipeline	16
11.3 Firmware Timing	16
12 Verification Plan	17
13 Known Limits and Engineering Assumptions	17
14 Appendix: Source Snippets	18
14.1 Snapshot Contract	18
14.2 Calibration Addresses	18
14.3 ADC Sequence	18

1 Executive Summary

This project is a bare-metal CMSIS firmware product for the NUCLEO-L476RG board. From the outside, the product behaves as a compact temperature monitor:

- It powers from the Nucleo board USB/ST-LINK path.
- It samples the STM32L476RG internal temperature sensor and internal voltage reference.
- It compensates the temperature reading against the measured analog supply.
- It displays the rounded temperature on a four-digit Multi-Function Shield display.
- It emits one diagnostic line per second through the ST-LINK virtual COM port.

The implementation is intentionally small. Hardware-facing side effects live in the driver modules under `src/`; the foreground application in `main.c` only sequences reads, display refresh, and serial reporting.

2 Source of Truth

The documentation is derived from the local firmware plus official STMicroelectronics documents:

- Local firmware: `src/main.c`, `src/temp_sensor.c`, `src/mfs_display.c`, `src/debug_uart.c`, and `src/timebase.c`.
- UM1724, *STM32 Nucleo-64 boards (MB1136)*, official ST user manual: https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf.
- MB1136-DEFAULT-C03 official board schematic: https://www.st.com/resource/en/schematic_pack/mb1136-default-c03_schematic.pdf.
- STM32L476xx official datasheet: <https://www.st.com/resource/en/datasheet/stm32l476re.pdf>.

Official document pages consulted for hardware evidence include the NUCLEO-L476RG Arduino connector table in UM1724 and sheet 3 of the MB1136 schematic, where the STM32 pin mapping, AVDD, USART pins, and Arduino digital signals are visible. The official PDFs are staged locally under `docs/source-of-truth/`; the screenshots in Section 10 are raster extracts from those documents. Project-specific diagrams are rendered as vector figures in this report.

3 Repository and Build Context

Item	Local project value
Core/domain path	<code>src/temp_sensor.c</code> , <code>include/temp_sensor.h</code>
Application/orchestration path	<code>src/main.c</code>
Display hardware adapter	<code>src/mfs_display.c</code> , <code>include/mfs_display.h</code>
Serial hardware adapter	<code>src/debug_uart.c</code> , <code>include/debug_uart.h</code>
Timing adapter	<code>src/timebase.c</code> , <code>include/timebase.h</code>
Build system	PlatformIO project, <code>platformio.ini</code>
Target board	<code>nucleo_l476rg</code>
Framework	CMSIS
Compiler flags	<code>-Wall -Wextra -Wpedantic</code>

Item	Local project value
Tests path	<code>test/README</code> ; no executable test suite is defined
Generated/vendor paths	<code>.pio/</code> build output is generated and should not be hand-edited

Canonical build command from the project README:

```
turn-signals/.venv/bin/pio run -d adc -e nucleo_l476rg
```

The current checkout does not contain that `pio` executable, and `pio` is not on `PATH`. The existing `.pio/build/` directory does contain prior firmware artifacts, but those artifacts are not a replacement for a fresh build.

4 Black-Box Product Contract

4.1 Inputs

- Electrical power through the Nucleo board USB/ST-LINK path.
- The STM32L476RG die temperature, sensed internally by the MCU.
- The MCU internal bandgap reference, `VREFINT`, used to infer the effective analog supply during conversion.

There is no external analog signal wired into this project. The ADC channels are internal channels selected by firmware.

4.2 Outputs

- Four-character Multi-Function Shield display text: `NN C`, `-N C`, `AERR`, or `DERR`.
- USART2 diagnostic line through the ST-LINK virtual COM port at 115200 baud.

Successful serial output has this contract:

```
adc-temp status=OK raw_temp=<adc_code> raw_vref=<adc_code> vdda_mv=<millivolts> temp_c=<signed_decimal_celsius>
```

Fault output has this contract:

```
adc-temp status=ERR adc_fault=<0_or_1> dma_fault=<0_or_1>
```

5 Hardware Architecture

5.1 Board and MCU

The target is the NUCLEO-L476RG board with an STM32L476RG microcontroller. UM1724 identifies the Nucleo-64 board family as Arduino Uno V3 compatible and exposes the STM32 pins through Arduino headers and ST morpho headers. The firmware uses these board-level connections:

Signal	STM32 pin	Use in this project
ADC1_IN17	internal	Internal temperature sensor analog signal.
ADC1_IN0	internal	VREFINT internal reference channel.
PA2	USART2_TX	ST-LINK virtual COM transmit path; Arduino D1 in UM1724.
PB5	D4	Multi-Function Shield latch pin in this firmware.

Signal	STM32 pin	Use in this project
PA8	D7	Multi-Function Shield shift clock pin in this firmware.
PA9	D8	Multi-Function Shield shift data pin in this firmware.
AVDD	board analog supply	ADC reference domain used by the conversion result.

The official MB1136 schematic shows the MCU sheet with PA2 labeled on the USART transmit net, PB5 mapped to D4, PA8 mapped to D7, PA9 mapped to D8, and VDDA/VREF+ tied into the analog supply domain through the board analog power network. Those connections match the firmware constants in `debug_uart.c` and `mfs_display.c`.

5.2 ADC Signal Path

The STM32L476xx datasheet describes the temperature sensor as an internal analog source connected to ADC1 channel 17 and ADC3 channel 17. The same datasheet describes VREFINT as an internal stable bandgap reference connected to ADC1 channel 0. The firmware selects ADC1 channel 17 followed by ADC1 channel 0 in the regular conversion sequence.

5.3 Display Interface

The display driver is a bit-banged serial interface using three GPIO outputs:

- latch: PB5, Arduino D4
- clock: PA8, Arduino D7
- data: PA9, Arduino D8

Each display refresh writes two bytes: inverted seven-segment bits and a digit select mask. The firmware multiplexes one digit every 2 ms, so each of four digits is refreshed every 8 ms, approximately 125 Hz per digit.

5.4 UART Interface

USART2 transmit uses PA2 alternate function 7. UM1724 maps PA2 to Arduino D1 and USART2_TX on the NUCLEO-L476RG connector table. The project uses transmit only; PA3/USART2_RX is not configured by this firmware.

6 Firmware Architecture

6.1 Module Responsibilities

Module	Responsibility
<code>main.c</code>	Board boot sequencing, display refresh cadence, serial reporting cadence, and sleep between interrupts.
<code>temp_sensor.c</code>	ADC1 and DMA1 Channel 1 setup, fault tracking, raw sample averaging, VDDA estimation, temperature calibration math.
<code>mfs_display.c</code>	GPIO setup for display pins, seven-segment character encoding, shift-register writes, digit blanking.

Module	Responsibility
<code>debug_uart.c</code>	USART2 TX GPIO alternate function setup and blocking transmit helpers.
<code>timebase.c</code>	SysTick millisecond counter.

6.2 Initialization Sequence

At boot, `main.c` executes:

1. `SystemCoreClockUpdate()` to synchronize CMSIS clock metadata.
2. `mfs_display_init()` to configure the display GPIOs and blank the display.
3. `timebase_init()` to start SysTick at 1 kHz.
4. `debug_uart_init(115200)` to enable the diagnostic channel.
5. `temp_sensor_init()` to configure ADC1, DMA1 Channel 1, internal ADC channels, calibration, and continuous conversion.

If ADC initialization fails, the serial channel reports a boot fault and the display path remains capable of showing `AERR`.

6.3 ADC and DMA Configuration

`temp_sensor_init()` configures a continuous two-channel ADC scan:

- ADC common register enables the ADC clock mode, temperature sensor, and VREFINT path.
- ADC self-calibration is run before enabling ADC1.
- DMA1 Channel 1 reads from ADC1->DR into `g_adc.samples`.
- DMA memory increment and circular mode are enabled.
- DMA peripheral and memory width are halfword.
- The regular sequence length is two conversions: temperature sensor channel 17 and VREFINT channel 0.
- ADC continuous conversion, DMA, DMA circular request mode, and overrun overwrite mode are enabled.

The sample buffer stores 16 repeated scan pairs:

$$N_{\text{pairs}} = 16, \quad N_{\text{channels}} = 2, \quad N_{\text{samples}} = 32.$$

For scan pair i , the buffer layout is:

$$x_{2i} = T_{\text{raw},i}, \quad x_{2i+1} = V_{\text{REFINT},i}.$$

7 Measurement and Calibration Equations

7.1 Averaging

The driver averages 16 raw samples for each channel:

$$\bar{T}_{\text{raw}} = \frac{1}{16} \sum_{i=0}^{15} T_{\text{raw},i}, \quad \bar{V}_{\text{ref}} = \frac{1}{16} \sum_{i=0}^{15} V_{\text{REFINT},i}.$$

The average is integer division in firmware. This intentionally trades fractional precision for a simple fixed-point implementation with no floating-point runtime dependency.

7.2 Analog Supply Estimation

ST stores a factory VREFINT calibration code measured at $V_{DDA,cal} = 3000\text{ mV}$. The firmware reads that halfword from `0x1FFF75AA`. Assuming the VREFINT physical voltage is stable, the ADC code is inversely proportional to the active ADC reference voltage. The firmware therefore estimates:

$$V_{DDA} = \frac{3000 \cdot VREFINT_{CAL}}{\overline{V}_{ref}} \text{ mV}.$$

This is exactly the integer expression in `temp_sensor.c`:

$$vdda_mv = \frac{VREFINT_CAL_MV \cdot vref_cal}{vref_raw}.$$

7.3 Temperature Code Compensation

The ST temperature calibration points are measured at 3.0 V. The active temperature ADC code is first normalized back to the 3.0 V calibration domain:

$$T_{code,3.0V} = \frac{\overline{T}_{raw} \cdot V_{DDA}}{3000}.$$

The firmware calls this value `temp_corrected`.

7.4 Two-Point Temperature Interpolation

The STM32L476xx datasheet gives two factory calibration points:

- `TS_CAL1` at address `0x1FFF75A8`, measured at 30 degrees C.
- `TS_CAL2` at address `0x1FFF75CA`, measured at 110 degrees C.

The firmware computes centi-degrees Celsius by linear interpolation:

$$T_{cC} = \left(\frac{(T_{code,3.0V} - TS_{CAL1}) \cdot (110 - 30) \cdot 100}{TS_{CAL2} - TS_{CAL1}} \right) + 30 \cdot 100.$$

The public snapshot field `temp_cent_i_c` stores this signed fixed-point result. For example, 2734 means 27.34 degrees C.

8 Runtime Behavior

8.1 Main Loop Cadence

The main loop is cooperative and time-triggered by the SysTick millisecond counter:

Cadence	Constant	Action
2 ms	<code>REFRESH_MS</code>	Scan the next display digit.
250 ms	<code>DISPLAY_UPDATE_MS</code>	Read latest ADC/DMA snapshot and refresh the display text buffer.
1000 ms	<code>SERIAL_UPDATE_MS</code>	Read latest snapshot and emit a UART diagnostic line.

After each loop pass, the firmware executes `_WFI()` and waits for the next interrupt. SysTick wakes the loop for normal timing. ADC overrun also has an interrupt path through `ADC1_2_IRQHandler()`.

8.2 Fault Model

The fault model is intentionally simple:

- ADC self-calibration timeout or ADC ready timeout sets `adc_fault`.
- ADC overrun interrupt sets `adc_fault`.
- DMA transfer error sets `dma_fault`.
- A snapshot is valid only when the sample path has data and neither fault flag is set.

The display contracts are:

- AERR: ADC path is invalid or not ready.
- DERR: DMA transfer path reported a fault.
- NN C or -N C: valid rounded temperature, clamped to the display range from -9 to 99 degrees C.

9 Connection Checklist

Connection	Source evidence	Practical check
PA2 / USART2_TX	UM1724 NUCLE0-L476RG connector table; MB1136 schematic	Serial terminal on ST-LINK VCP should receive one line per second at 115200 baud.
PB5 / D4	UM1724 connector table; MB1136 schematic; <code>mfs_display.c</code>	Display latch transitions during refresh.
PA8 / D7	UM1724 connector table; MB1136 schematic; <code>mfs_display.c</code>	Display clock toggles during refresh.
PA9 / D8	UM1724 connector table; MB1136 schematic; <code>mfs_display.c</code>	Display data changes before clock pulses.
ADC1 channel 17	STM32L476xx datasheet; <code>temp_sensor.c</code>	Raw temperature code appears in serial telemetry.
ADC1 channel 0	STM32L476xx datasheet; <code>temp_sensor.c</code>	Raw VREFINT code is nonzero; derived VDDA is plausible.

10 Official Hardware Evidence

The hardware claims in this report are tied to the official ST documents listed in Section 2. The following screenshots are direct page renders from the staged official PDFs. The explanation below each screenshot states exactly which firmware claim the official page supports.

10.1 NUCLEO-L476RG Arduino Connector Evidence

UM1724

Hardware layout and configuration

Table 22. ARDUINO® connectors on NUCLEO-L452RE (continued)

Connector	Pin	Pin name	STM32 pin	Function
CN9 digital	7	D6	PB10	TIM2_CH3
	6	D5	PB4	TIM3_CH1
	5	D4	PB5	-
	4	D3	PB3	TIM2_CH2
	3	D2	PA10	-
	2	D1	PA2	USART2_TX
	1	D0	PA3	USART2_RX

1. Refer to [Table 10: Solder bridges](#) for details.

Table 23. ARDUINO® connectors on NUCLEO-L476RG

Connector	Pin	Pin name	STM32 pin	Function
Left connectors				
CN6 power	1	NC	-	-
	2	IOREF	-	3.3V Ref
	3	RESET	NRST	RESET
	4	+3.3V	-	3.3V input/output
	5	+5V	-	5V output
	6	GND	-	ground
	7	GND	-	ground
	8	VIN	-	Power input
CN8 analog	1	A0	PA0	ADC12_IN5
	2	A1	PA1	ADC12_IN6
	3	A2	PA4	ADC12_IN9
	4	A3	PB0	ADC12_IN15
	5	A4	PC1 or PB9 ⁽¹⁾	ADC123_IN2 (PC1) or I2C1_SDA (PB9)
	6	A5	PC0 or PB8 ⁽¹⁾	ADC123_IN1 (PC0) or I2C1_SCL (PB8)
Right connectors				
CN5 digital	10	D15	PB8	I2C1_SCL
	9	D14	PB9	I2C1_SDA
	8	AREF	-	AVDD
	7	GND	-	ground
	6	D13	PA5	SPI1_SCK
	5	D12	PA6	SPI1_MISO
	4	D11	PA7	TIM17_CH1 or SPI1_MOSI
	3	D10	PB6	TIM4_CH1 or SPI1_CS



UM1724 Rev 17

53/91

Figure 1: UM1724 page 53, where Table 23 begins for the NUCLEO-L476RG Arduino connectors. This page shows AREF as AVDD and begins the connector table used to validate the board-level signal naming.

Table 23. ARDUINO® connectors on NUCLEO-L476RG (continued)

Connector	Pin	Pin name	STM32 pin	Function
CN5 digital	2	D9	PC7	TIM3_CH2
	1	D8	PA9	-
CN9 digital	8	D7	PA8	-
	7	D6	PB10	TIM2_CH3
	6	D5	PB4	TIM3_CH1
	5	D4	PB5	-
	4	D3	PB3	TIM2_CH2
	3	D2	PA10	-
	2	D1	PA2	USART2_TX
	1	D0	PA3	USART2_RX

1. Refer to [Table 10: Solder bridges](#) for details.

7.14 ST morpho connector

The ST morpho connector consists of male pin headers (CN7 and CN10) accessible on both sides of the board. They can be used to connect the STM32 Nucleo board to an extension board or a prototype/wrapping board placed on the top or bottom side of the STM32 Nucleo board. All signals and power pins of the STM32 are available on the ST morpho connector. This connector can also be probed by an oscilloscope, logical analyzer, or voltmeter.

[Table 24](#) to [Table 33](#) show the pin assignments of each STM32 on the ST morpho connector.

Table 24. ST morpho connector on NUCLEO-F030R8

CN7 odd pins		CN7 even pins		CN10 odd pins		CN10 even pins	
Pin	Name	Name	Pin	Pin	Name	Name	Pin
1	PC10	PC11	2	1	PC9	PC8	2
3	PC12	PD2	4	3	PB8	PC6	4
5	VDD	E5V	6	5	PB9	PC5	6
7	BOOT0 ⁽¹⁾	GND	8	7	AVDD	U5V ⁽²⁾	8
9	PF6	-	10	9	GND	-	10
11	PF7	IOREF	12	11	PA5	PA12	12
13	PA13	RESET	14	13	PA6	PA11	14
15	PA14	+3.3V	16	15	PA7	PB12	16
17	PA15	+5V	18	17	PB6	PB11	18
19	GND	GND	20	19	PC7	GND	20
21	PB7	GND	22	21	PA9	PB2	22

Figure 2: UM1724 page 54, continuation of Table 23. This screenshot shows the actual firmware pins: D8 maps to PA9, D7 maps to PA8, D4 maps to PB5, and D1 maps to PA2 with USART2_TX. These are the display data, display clock, display latch, and serial transmit connections used by the project.

10.3 STM32L476xx Internal ADC Evidence

3.15 Analog to digital converter (ADC)

The device embeds 3 successive approximation analog-to-digital converters with the following features:

- 12-bit native resolution, with built-in calibration
- 5.33 Msp/s maximum conversion rate with full resolution
 - Down to 18.75 ns sampling time
 - Increased conversion rate for lower resolution (up to 8.88 Msp/s for 6-bit resolution)
- Up to 24 external channels, some of them shared between ADC1 and ADC2, or ADC1, ADC2 and ADC3.
- 5 internal channels: internal reference voltage, temperature sensor, VBAT/3, DAC1_OUT1 and DAC1_OUT2.
- One external reference pin is available on some package, allowing the input voltage range to be independent from the power supply
- Single-ended and differential mode inputs
- Low-power design
 - Capable of low-current operation at low conversion rate (consumption decreases linearly with speed)
 - Dual clock domain architecture: ADC speed independent from CPU frequency
- Highly versatile digital interface
 - Single-shot or continuous/discontinuous sequencer-based scan mode: 2 groups of analog signals conversions can be programmed to differentiate background and high-priority real-time conversions
 - Handles two ADC converters for dual mode operation (simultaneous or interleaved sampling modes)
 - Each ADC support multiple trigger inputs for synchronization with on-chip timers and external signals
 - Results stored into 3 data register or in RAM with DMA controller support
 - Data pre-processing: left/right alignment and per channel offset compensation
 - Built-in oversampling unit for enhanced SNR
 - Channel-wise programmable sampling time
 - Three analog watchdog for automatic voltage monitoring, generating interrupts and trigger for selected timers
 - Hardware assistant to prepare the context of the injected channels to allow fast context switching

3.15.1 Temperature sensor

The temperature sensor (TS) generates a voltage V_{TS} that varies linearly with temperature.

The temperature sensor is internally connected to the ADC1_IN17 and ADC3_IN17 input channels which is used to convert the sensor output voltage into a digital value.

The sensor provides good linearity but it has to be calibrated to obtain good overall accuracy of the temperature measurement. As the offset of the temperature sensor varies from chip to chip due to process variation, the uncalibrated internal temperature sensor is suitable for applications that detect temperature changes only.



Figure 4: STM32L476xx datasheet page 41. The ADC functional overview lists the internal reference and temperature sensor among the ADC internal channels, then states that the temperature sensor is internally connected to ADC1_IN17 and ADC3_IN17. The firmware uses ADC1 channel 17 for this reading.

To improve the accuracy of the temperature sensor measurement, each device is individually factory-calibrated by ST. The temperature sensor factory calibration data are stored by ST in the system memory area, accessible in read-only mode.

Table 8. Temperature sensor calibration values

Calibration value name	Description	Memory address
TS_CAL1	TS ADC raw data acquired at a temperature of 30 °C (± 5 °C), $V_{DDA} = V_{REF+} = 3.0$ V (± 10 mV)	0x1FFF 75A8 - 0x1FFF 75A9
TS_CAL2	TS ADC raw data acquired at a temperature of 110 °C (± 5 °C), $V_{DDA} = V_{REF+} = 3.0$ V (± 10 mV)	0x1FFF 75CA - 0x1FFF 75CB

3.15.2 Internal voltage reference (V_{REFINT})

The internal voltage reference (V_{REFINT}) provides a stable (bandgap) voltage output for the ADC and Comparators. V_{REFINT} is internally connected to the ADC1_IN0 input channel. The precise voltage of V_{REFINT} is individually measured for each part by ST during production test and stored in the system memory area. It is accessible in read-only mode.

Table 9. Internal voltage reference calibration values

Calibration value name	Description	Memory address
V_{REFINT}	Raw data acquired at a temperature of 30 °C (± 5 °C), $V_{DDA} = V_{REF+} = 3.0$ V (± 10 mV)	0x1FFF 75AA - 0x1FFF 75AB

3.15.3 V_{BAT} battery voltage monitoring

This embedded hardware feature allows the application to measure the V_{BAT} battery voltage using the internal ADC channel ADC1_IN18 or ADC3_IN18. As the V_{BAT} voltage may be higher than V_{DDA} , and thus outside the ADC input range, the V_{BAT} pin is internally connected to a bridge divider by 3. As a consequence, the converted digital value is one third the V_{BAT} voltage.

3.16 Digital to analog converter (DAC)

Two 12-bit buffered DAC channels can be used to convert digital signals into analog voltage signal outputs. The chosen design structure is composed of integrated resistor strings and an amplifier in inverting configuration.

Figure 5: STM32L476xx datasheet page 42. This page shows the temperature calibration words TS_CAL1 and TS_CAL2, and states that V_{REFINT} is connected to ADC1_IN0. The same page shows the V_{REFINT} calibration word used by the firmware to estimate V_{DDA} before applying the temperature equation.

11 System Diagrams

11.1 Product Context

Figure 6 shows the black-box product boundary. The user only sees power, the display, and the serial diagnostic stream. The temperature sensor and VREFINT source are internal MCU resources, so the external hardware contract is small: the Nucleo board, the display shield pins, and the ST-LINK serial path.

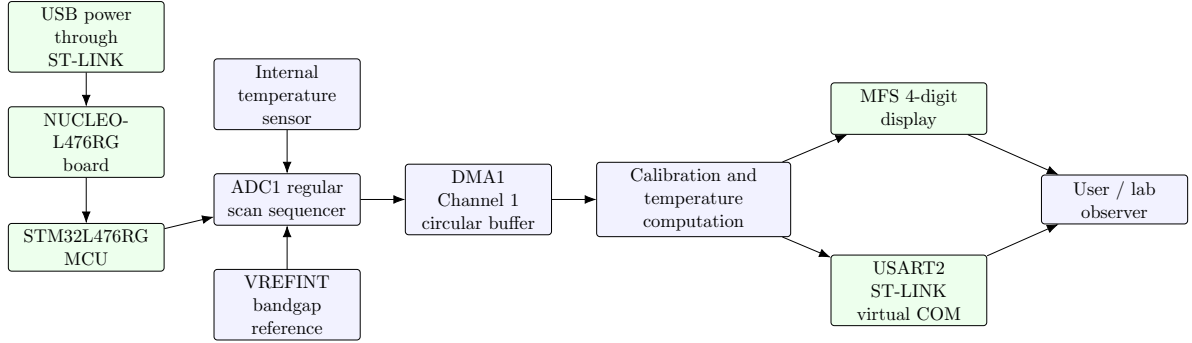


Figure 6: Product-level context from power input to human-visible outputs.

11.2 ADC and DMA Pipeline

Figure 7 shows the measurement path implemented in `temp_sensor.c`. Initialization prepares the ADC regulator, common ADC internal channels, self-calibration, DMA, and regular scan sequence. Runtime data then flows from ADC1 into the circular DMA buffer, through averaging, VDDA estimation, temperature interpolation, and finally to the display and UART reporting paths. ADC overrun and DMA transfer errors bypass the normal valid snapshot path and are surfaced as fault states.

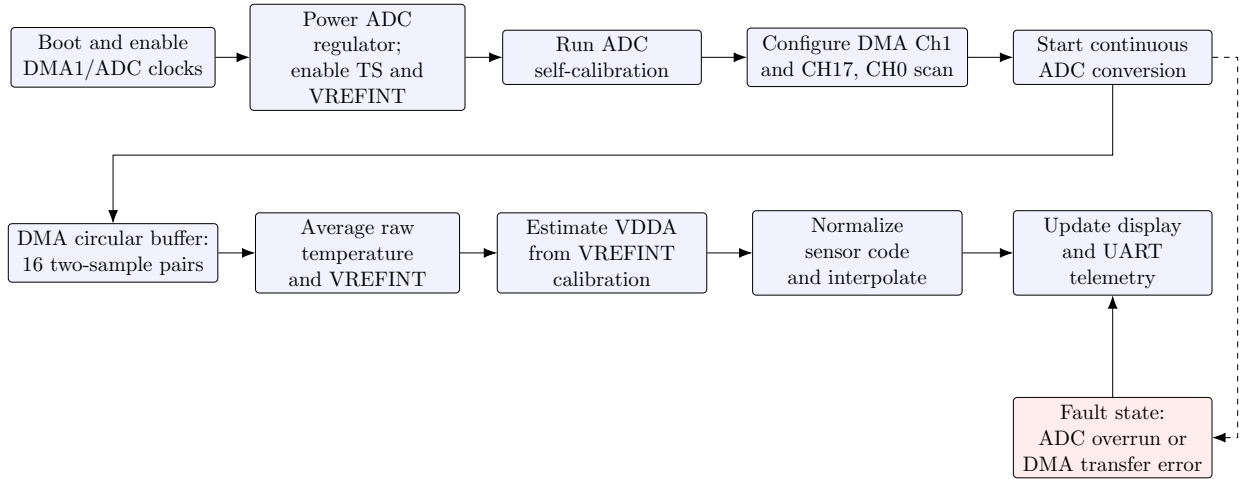


Figure 7: ADC1, DMA1 Channel 1, calibration, and output pipeline.

11.3 Firmware Timing

Figure 8 summarizes the cooperative timing model. The SysTick interrupt owns the millisecond counter; the foreground loop checks elapsed time and performs short scheduled actions. Display multiplexing runs every 2 ms, display text updates every 250 ms, and serial diagnostics run every 1000 ms. Between checks, the firmware waits for interrupt with `__WFI()`.

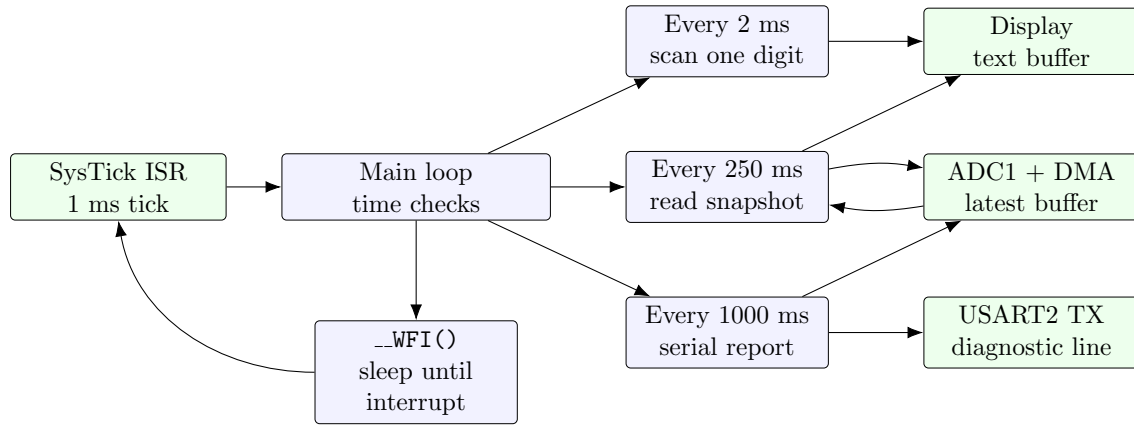


Figure 8: Foreground timing and interrupt wake-up model.

12 Verification Plan

The expected engineering validation path is:

1. Build with PlatformIO using the project environment `nucleo_l476rg`.
2. Flash the board over ST-LINK.
3. Open the virtual COM port at 115200 baud.
4. Confirm boot line:

```
adc-temp boot: ADC1 channel 17 + VREFINT, DMA1 channel 1 circular
```

5. Confirm repeated `status=OK` lines with nonzero `raw_temp`, nonzero `raw_vref`, plausible `vdda_mv`, and plausible room-temperature `temp_c`.
6. Confirm the display shows rounded Celsius and does not remain on AERR or DERR.

This documentation package itself is validated by compiling `adc-temperature-monitor.tex` with `pdflatex`.

13 Known Limits and Engineering Assumptions

- The internal temperature sensor measures MCU die temperature, not ambient air temperature.
- The conversion assumes the ST factory calibration words are readable and valid for the target STM32L476RG device.
- The firmware uses integer arithmetic. This is deterministic and small, but it truncates intermediate fractional values.
- The display is limited to four characters, so the human-facing display clamps valid values to -9 through 99 degrees C.
- UART transmit is blocking. At one line per second this is acceptable for the current product contract.
- No hardware-in-the-loop validation is encoded in the repository test tree. Board validation remains a manual upload and serial/display check.

14 Appendix: Source Snippets

14.1 Snapshot Contract

```
typedef struct {  
    uint16_t temp_raw;  
    uint16_t vref_raw;  
    uint32_t vdda_mv;  
    int32_t temp_cent_i_c;  
    bool valid;  
    bool adc_fault;  
    bool dma_fault;  
} temp_sensor_snapshot_t;
```

14.2 Calibration Addresses

```
TS_CAL1_ADDR = 0x1FFF75A8UL  
TS_CAL2_ADDR = 0x1FFF75CAUL  
VREFINT_CAL_ADDR = 0x1FFF75AAUL
```

14.3 ADC Sequence

```
ADC1->SQR1 = (1UL << ADC_SQR1_L_Pos)  
            | (ADC_CHANNEL_TEMPSENSOR << ADC_SQR1_SQ1_Pos)  
            | (ADC_CHANNEL_VREFINT << ADC_SQR1_SQ2_Pos);
```